

CostSphere: A Cost Model-Driven Privacy-Preserving Machine Learning Framework with Network Context Adaptation

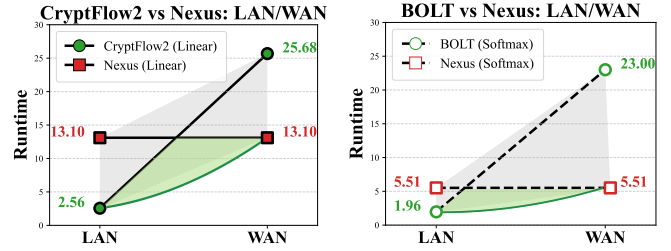
Yuntian Chen, Zhanyong Tang, Tianpei Lu, Bingsheng Zhang, Zhiying Shi, Zhiyuan Ning

Privacy-preserving machine learning (PPML) is critical for protecting sensitive data in domains like healthcare, finance, and recommendation systems. Fully Homomorphic Encryption (FHE) and Secure Multi-Party Computation (MPC) are key enablers of secure computation, yet existing hybrid approaches often suffer from fixed protocol assignments, resulting in inefficiencies across diverse network environments, such as LANs and WANs. To address this, we introduce COSTSPHERE, a cost-model-driven framework that dynamically assigns FHE and MPC protocols to optimize computational efficiency under varying network conditions. Utilizing a predictive cost model based on MLIR’s TOSA-level dialect and an ILP-based solver, COSTSPHERE ensures robust performance for Transformer-based models. Experimental results demonstrate that COSTSPHERE delivers $6.68\times$ to $12.92\times$ improvements in inference runtime compared to state-of-the-art solutions like BumbleBee (NDSS ’25), enabling scalable and network-agnostic PPML across diverse computational scenarios.

I. INTRODUCTION

The widespread adoption of machine learning across domains such as healthcare, finance, and recommendation systems [1]–[3] has intensified concerns about data privacy, particularly when handling sensitive personal information [4]. To address these concerns, Privacy-Preserving Machine Learning (PPML) [5], [6], [7], [8], [9] has emerged as a critical field, leveraging techniques like Secure Multi-Party Computation (MPC) [10] and Fully Homomorphic Encryption (FHE) [11], [12]. MPC enables n participants, each with private inputs $x_1, \dots, x_n$, to compute a function $f(x_1, \dots, x_n)$ while revealing only the output. FHE allows computations on encrypted data, producing encrypted results that can be decrypted only by the private key holder, ensuring equivalence to plaintext computations without data leakage.

In practical PPML applications, particularly for complex models like Transformers [13], computations involve both large-scale linear operations (e.g., matrix multiplications) and non-linear operations (e.g., SoftMax, Tanh, ReLU) [14]. Efficiently managing these diverse operations requires the synergistic use of MPC and FHE to exploit their complementary strengths. Recent studies have explored manually optimized hybrid protocols to enhance computational efficiency in PPML, such as ABY [15], ABY3 [16], CostCo [17], and [18]–[21], addressing the challenges posed by complex computational loads and non-linear operations. However, these



(a) Single technique for Matrix multiplication. (b) Single technique for Softmax function.
Fig. 1: Comparison of the performance of a single technology in different deployment environments. WAN={100mbps bandwidth, 80ms latency}. The green markings indicate the use of MPC technology, while the red ones indicate the use of FHE technology.

approaches often lack adaptability to varying network conditions, limiting their effectiveness in dynamic environments.

Current research on privacy-preserving inference for Transformer-based models, such as BERT [22] and GPT [23], often employs a fragmented approach to assigning Fully Homomorphic Encryption (FHE) and Multi-Party Computation (MPC) protocols. Works like BOLT [24] and BumbleBee [25] categorize operators into linear operations (e.g., matrix multiplications) and non-linear operations (e.g., activation functions such as SoftMax, GeLU, and LayerNorm). Linear operations are typically assigned to FHE protocols, while non-linear operations are delegated to MPC protocols. This rigid, operator-specific protocol assignment lacks adaptability, leading to suboptimal performance across varying network conditions, such as Local Area Networks (LANs) and Wide Area Networks (WANs).

For instance, in matrix multiplication, CryptFlow2 [26] leverages MPC-based techniques, achieving superior performance over FHE-based Nexus [27] in LAN environments due to lower communication overhead. However, as shown in Figure 1a, CryptFlow2’s performance degrades significantly in WAN settings, where increased latency and reduced bandwidth amplify communication costs, rendering it less efficient than Nexus. Similarly, for non-linear operations like SoftMax, Nexus approximates the function using FHE, while BOLT [24] relies on MPC. As illustrated in Figure 1b, BOLT outperforms Nexus in LAN environments, but its reliance on frequent communication in MPC leads to substantial performance degrada-

tion in WAN settings, where Nexus’s FHE-based approach proves more resilient.

This fixed protocol assignment results in inconsistent performance, particularly in the transitional **gray areas** of network conditions, as depicted in Figures 1a and 1b. These inefficiencies stem from the inherent limitations of assigning a single protocol to each operator without considering network dynamics. Dynamic hybrid protocol performance, represented by the **green areas** in the figures, remains unattainable with current approaches, as they fail to dynamically adapt protocol assignments to network variability. This lack of flexibility underscores a critical challenge: achieving robust, network-agnostic privacy-preserving inference requires a more integrated and adaptive protocol assignment strategy, capable of balancing computational and communication costs across diverse network environments.

In response to this challenge, we introduce a hybrid protocol assignment methodology that dynamically reassigns FHE and MPC protocols to operators with the objective of attaining the optimal inference performance, as depicted by **green line** in Figure 1. A key step in this approach is to accurately quantify the execution time of each protocol under various configurations of parameters and network conditions. Given the large number of operators in complex Transformer-based models, exhaustively testing each operator across all possible configurations is computationally prohibitive. We therefore propose a predictive cost model that estimates protocol overhead based on operator characteristics and network parameters. By integrating this cost model with an optimization solver, we can systematically determine the optimal protocol allocation for diverse network environments, thereby enhancing adaptability and minimizing performance degradation across the spectrum of network conditions.

Despite the promise of hybrid protocol assignment, existing strategies suffer from several critical limitations that hinder their ability to achieve such optimal performance.

Heavy Customization Effort and Limited Generality. Current cost models, such as those in [28] and [29], rely on manually defining the costs of basic operators (e.g., $+$, $-$, $\times$, $\div$, $\geq$, $\gg$, and $\ll$) to customize the cost of composite operators. This labor-intensive process is extremely time-consuming. Additionally, there is a generality issue in protocol assignment across different technologies. For example, methods like ABY [16], CostCO [17], and Silph [30] primarily focus on protocol assignment for MPC technology. However, for some complex computational tasks, FHE is more advantageous in scenarios with poor network conditions. Currently, there is a lack of a hybrid protocol assignment strategy that addresses different cryptographic technologies.

Static Constraint Solvers Lacking Dynamic Adaptability. Current constraint solvers, such as CostCO [17], Silph [30], and Floria et.al, [31] are designed for a single cryptographic technique. Introducing different cryptographic techniques requires domain conversions, which often introduce communication and computational costs. Frequent conversions may result in higher overall costs. However, existing solutions

fail to manage these conversions effectively. Moreover, current solutions cannot handle real-time changes in protocol costs during the assignment process. In FHE computation, dynamic cost changes occur during protocol assignment for a sequence of operators. This is because the depth of homomorphic multiplication is fixed in advance, but setting different multiplication depths requires changing encryption parameters, which leads to changes in homomorphic computation costs. During the protocol selection process, if homomorphic multiplication is continuously selected, the multiplication depth will increase, requiring a global update of homomorphic computation costs.

To overcome these limitations, we present COSTSPHERE, a novel framework for hybrid protocol assignment that integrates a cost model-driven approach with a dynamic optimization solver, as depicted in Figure 2. COSTSPHERE enables efficient private inference for Transformer-based models by adapting to diverse network environments while minimizing inference time. Our approach leverages MLIR’s TOSA-level dialect for a unified operator representation, coupled with a Multi-Output training method to generate independent, high-precision cost models for each operator. These models, tailored to FHE and MPC, eliminate manual cost estimation. For protocol assignment, we employ an Integer Linear Programming (ILP)-based solver that dynamically updates FHE costs based on homomorphic multiplication depth, incorporating a penalty strategy to account for increasing costs and a constraint to manage FHE-MPC transition overhead. Additionally, homomorphic bootstrapping costs are integrated as a constraint when multiplication depth reaches a preset threshold, further optimizing the assignment process.

Specifically, we make the following key contributions.

- **High-precision cost prediction model:** We’re the first to employ MLIR’s TOSA-level dialect as a unified operator interface to develop a cost prediction model. The coefficient of determination (R^2) value [32] reaches 95% to 99%. This model offers the flexibility to be used either independently or in a hybrid manner for any complex MPC and FHE applications.
- **Dynamic Protocol Assignment:** We establish an ILP-based optimization model for hybrid protocol assignment, employing two constraints to balance continuous same-protocol assignment with cross-protocol conversion, and dynamically to update FHE costs based on homomorphic multiplication depth.
- **Scalable Network-Adaptive Private Computing Framework:** We build an end-to-end framework COSTSPHERE and leverage COSTSPHERE to provide an efficient private inference solution for BERT models. Compared to BumbleBee, the communication volume is reduced by 18.59 GB, and the inference time is improved by 6.68 to 12.92 times.

II. PRELIMINARIES AND BACKGROUND

A. Common symbols

We provide a description of common symbols, as shown in Table I.

TABLE I: A table of notations for the mathematical symbols used

Notation	Description
$\mathcal{P}$	Participant
$\mathcal{A}$	Alice (Client)
$\mathcal{B}$	Bob (Server)
$\mathbf{x}/\mathbf{X}$	Vector/Matrix
$\langle m \rangle_{\mathcal{P}} / \langle m \rangle_{\mathcal{P}}^B$	Additive/Boolean sharing of m held by $\mathcal{P}$
$\llbracket m \rrbracket_{\mathcal{P}}$	The message encrypted by participant $\mathcal{P}$
Ω	Representation of the Meta-Op
FSOP	FHE-based specific protocol
MSOP	MPC-based specific protocol
Π	Specific protocol implementation
$\hat{C}$	The cost of the protocol
CM	Indicate the cost model, such $\text{CM}_{MPC}(\Omega_i)$

B. FHE

Fully Homomorphic Encryption (FHE) is first proposed by Rivest et al. [11]. We chose a variant of the Cheon-Kim-Kim-Song (CKKS) scheme [33], specifically the Residue Number System-based CKKS (RNS-CKKS) [34]. Whose security is based on the Ring Learning With Errors (RLWE) problem proposed by Lyubashevsky et al. [35]. The RNS-CKKS scheme comprises several core algorithms, including **KeyGen**, **Encrypt**, **Decrypt**, **HAdd**, **HMult** and **Square**. **KeyGen** generates the keys used in the FHE scheme given the parameters chosen. **Encrypt** and **Decrypt** are the encryption and decryption algorithms respectively. **HAdd**, **HMult**, **Square** denote homomorphic addition, homomorphic multiplication operations, and homomorphic squaring operations, respectively. A formal description with complete details can be found in [33].

C. MPC

MPC guarantees data privacy and correctness while eliminating needing a trusted third party. It encompasses various techniques, including Oblivious Transfer (OT) [36], Look-Up Tables (LUTs) [37], and Secret Sharing (SS) [38], [16]. This paper focuses on SS, which allowing a set of 2 participants $\mathcal{P} \in \{\mathcal{A}, \mathcal{B}\}$ to share a piece of information in such a way that only authorized subsets of the participants can recover the secret. Then we have the **Additive Scheme**: For every secret $s \in \mathbb{Z}_{2^k}$, an additive scheme satisfies $s = \langle s \rangle_{\mathcal{A}} + \langle s \rangle_{\mathcal{B}} \bmod 2^k$. When $k = 1$, the boolean secret sharing of s can be represented as $\langle s \rangle_{\mathcal{P}}^B$, satisfying $s = \langle s \rangle_{\mathcal{A}}^B \wedge \langle s \rangle_{\mathcal{B}}^B \bmod \mathbb{Z}_2$.

D. Tensor Operator Set Architecture

Tensor Operator Set Architecture (TOSA) [39] provides a set of whole-tensor operations commonly employed by Deep Neural Networks. TOSA categorizes its operator set into classes and attempts to address major functional operations at the tensor level, including compute, reduction, elementwise transformations, comparison, and control flow.¹ Most operators from the common ML frameworks (TensorFlow, PyTorch, etc.) should be expressible in TOSA.

¹<https://mlir.llvm.org/docs/Dialects/TOSA>

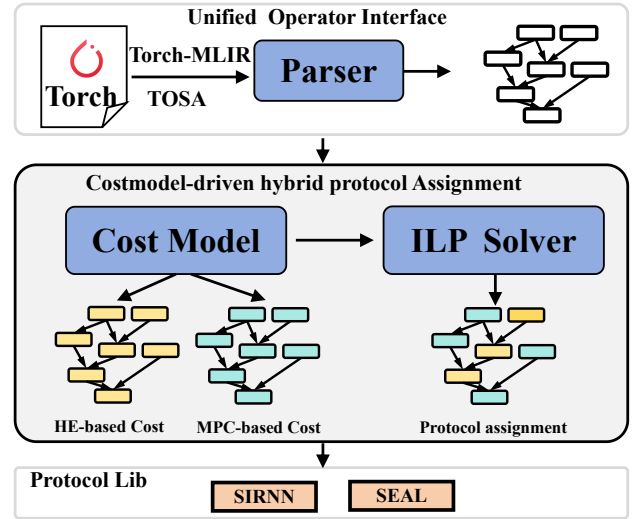


Fig. 2: The overview of COSTSPHERE

E. Thread model

The fundamental protocols presented in this paper are executed directly between the Client and Server, and are secure under the semi-honest threat model [19]. In COSTSPHERE, we employ advanced underlying protocols proven to be semi-honest secure [40].

III. COSTSPHERE OVERVIEW

COSTSPHERE is an end-to-end automated private inference framework designed for network environment adaptability. It includes Meta-operations (Meta-ops) (III-A), Cost Model (CM) (III-B), and Protocol Assignment (PAS) (III-C).

A. Meta Operators

To achieve upward compatibility with different neural network models and downward compatibility with various cryptographic techniques, the primary step is to model operators to unify the operator interface of neural networks. This involves two key considerations: (1) The operator needs to be compatible with mainstream neural network models; (2) The decomposition of the model into basic operators should enable a one-to-one correspondence with underlying cryptographic protocols, thereby achieving seamless integration between the model and privacy-preserving computation protocols.

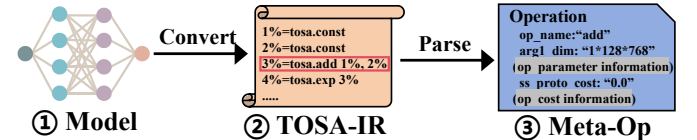


Fig. 3: Operator Modeling: From ML models to Operator-friendly Cryptographic Primitives Meta-Op (It is denoted as $\Omega_i = (op, C_i)$), where "op" represents the operator in Meta-ops sequence, and " C_i " records the input and output information as well as the context information of the operator.

Existing solutions like CostCO [17] focus on MPC cost modeling by using custom compilers CirC [41] to compile the computations into circuit operations (e.g., AND, XOR) and predict costs based on circuit features. While effective for circuit-level MPC protocols like Garbled Circuits (GC) [42]. Although building cost models at the circuit level can achieve high accuracy, it suffers from high computational complexity and low efficiency, making it unsuitable for cost estimation of large-scale deep learning models. Moreover, circuit-level operators are difficult to unify across different cryptographic techniques, limiting the model’s generality and scalability.

To overcome the limitations of circuit-level modeling and enable cost estimation across heterogeneous cryptographic protocols. We first adopt the TOSA dialect (graph-level operator representation) [39] in MLIR to describe computation tasks for cost modeling. Specifically, we utilize the torch-mlir compiler [43] to transform PyTorch-defined neural networks into MLIR, which is subsequently lowered to TOSA-level operators. These operators offer a uniform, hardware- and protocol-agnostic abstraction of computation. We then parse the TOSA operators into Meta-ops in Static Single Assignment (SSA) form, which serve as the foundational unit for constructing protocol-specific cost models.

TABLE II: Unified Description of a Meta-Op’s Paradigm, is denoted as C_l (Such as *tosa.Matmul op*)

Category	Field	Value / Example
Core Operator Info	Operator Name	MatrixMul
	Operator ID	OP_MATMUL_001
Input Parameters	Input 1 Details	
	Parameter ID	%arg0
	Dimension	(128, 768)
	Is_Const	FALSE
Input Parameters	Input 2 Details	
	Parameter ID	%20
	Dimension	(768, 768)
	Is_Const	True
Output Parameters	Output 1 Details	
	Parameter ID	output_matrix_C
	Dimension	(128, 768)
	Is_const	FALSE
Cost Information	HE Protocol Cost	0.0 ms
	MPC Protocol Cost	0.0 ms
	HE Bootstrapping Cost	0.0 ms
	Protocol Conversion Cost	0.0 ms

The specific formalization transformation process is as follows. Define a set of neural network operators $\mathcal{O} = \{O_1, O_2, \dots, O_l\}$, where each O_i represents an operation (e.g., reduce_sum, mul, add, rsqrt). The goal is to convert $\mathcal{O}$ into a unified intermediate representation that is compatible with both neural networks and cryptographic protocols.

The process starts by generating the MLIR representation of the model using torch-mlir. This is then converted into the TOSA dialect and parsed into Meta-Ops ($M\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_k\}$), as shown in Figure 3. Each Meta-Op $\Omega_l = (\text{op}, C_l)$ consists of an abstract operation (op), such as `op.add`, and a computational feature vector (C_l), which includes details like input dimensions and data types, see Table II.

For example, for the softmax function in the Attention mechanism, the computation is as follows:

$$y = \text{SoftMax}(x_i) = \frac{\exp(x_i - \bar{x})}{\sum_j \exp(x_j - \bar{x})}. \quad (1)$$

where $\bar{x}$ is the max value of the input vector². Through conversion, TOSA IR can be obtained. Then, after parsing the lowered TOSA IR, we obtain Meta-OPs that serve as a unified interface to map to operators of different cryptographic techniques, as shown in the following listing:

TABLE III: Meta-OPs of SoftMax

```
func.func @softmax(input [12x128x128]) ->
  output [12x128x128] {
    %50 = op.max, C_1[50]
    %51 = op.sub, C_1[51]
    %52 = op.exp, C_1[52]
    %53 = op.sum, C_1[53]
    %54 = op.reciprocal, C_1[54]
    %55 = op.mul, C_1[55]
    return %55[12x128x128] }
```

Through the Meta-Op interface, we can obtain operator sequences adapted for both FHE and MPC protocols. These sequences include operator information and cost-related details (e.g., `he_proto_cost`) reserved for the subsequent cost model. It is worth noting that computationally intensive (e.g., add, mul) operators in MLIR can be naturally adapted, but memory-intensive operators, such as select, need to be parsed into the *if_else* protocol in MPC. The detailed analysis process is presented in Section IV-A.

B. Cost Model

This section introduces the Cost Model (CM), a performance overhead model designed to predict the runtime of privacy inference protocols. The workflow of CM is shown in Figure 4. In the design of the CM, we first adopted a strategy of technology decoupling, followed by operator independence.

Specifically, ① for the technology decoupling, this is because FHE and MPC technologies differ significantly. For FHE, cost-influencing features include not only input dimensions, but also homomorphic encryption parameters like `poly_degree` and supported multiplication depth. For MPC, features affecting cost include ring size, network bandwidth, and network latency. Due to these disparities, we’ve decoupled FHE and MPC, designing separate cost models for each. ② For the operator Independence: We observed that even within the same technology (e.g., MPC’s mul protocol versus its rsqrt protocol), the computational costs vary greatly. Therefore, using common one-hot encoding for modeling would lead to reduced accuracy due to these inter-operator cost differences. To address this, we adopted an operator-independent strategy,

²In the original Torch implementation (<https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html>), the softmax function does not require subtracting $\bar{x}$. However, in FHE and MPC, subtracting $\bar{x}$ is necessary to ensure that all inputs to the exponential function are non-positive, thereby achieving numerical stability. [44], [27]

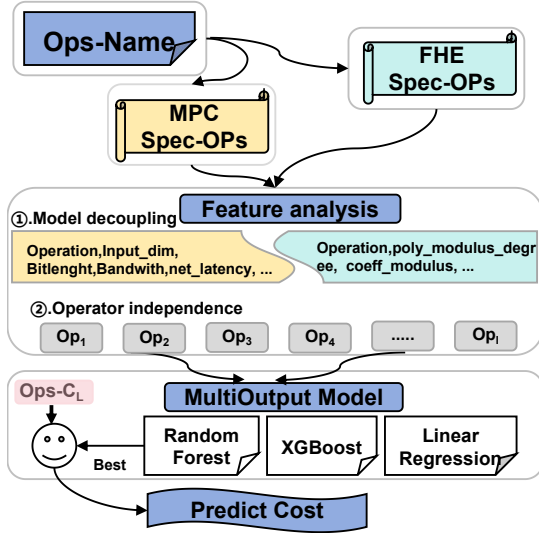


Fig. 4: CM workflow. We can obtain the datasets required for the feature analysis by running the Meta-op protocol.

building a cost model for each operator using a multi-output approach.

The benefit of this approach is that: (1). The cost model for FHE (CM_{FHE}) can be applied to any homomorphic computation task based on RNS-CKKS. Similarly, the MPC cost model (CM_{MPC}) can be applied to any 2-PC MPC task. (2). This design offers high flexibility and scalability. If an underlying protocol changes, we only need to modify the cost model corresponding to that specific protocol, and don't have to alter the entire CM system. Likewise, new protocols can be directly integrated into the CM system.

For the training process, the CM utilizes the operator name from Meta-Ops to match it with the corresponding MPC and FHE protocols. It then sets up extensive data dimensions, encryption parameters, and network environment settings to generate training data for the regression models. Based on these data, three different regression models (Random Forest, XGBoost, Linear Regression) [45] are trained simultaneously, and the best-performing model is selected as the cost model for MSOP/FSOP prediction. For the prediction process, in COSTSPHERE, upon receiving Meta-Ops, the CM determines the parameter configurations for specific FHE/MPC protocols based on the operator's parameter information and the characteristic features of both technologies, thereby obtaining the predicted runtime. When generating the predicted runtime, we incorporated a judgment based on the operator's input parameter types. Specifically, when an input parameter is a constant (i.e., $Is_const = True$ in C_L), the FHE cost we provide is the ciphertext-plaintext computation time. Otherwise, the predicted time is for ciphertext-ciphertext computation. This is because, in FHE, ciphertext-ciphertext operations for the same computation are more time-consuming than ciphertext-plaintext operations [46].

The specific overview of the CM is as follows: Given the

set of Meta-Ops $M\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_k\}$ obtained from the previous steps, the CM system operates as follows:

- **Protocol Mapping & Feature Analysis.** For each Meta-Op $\Omega_l = (op, C_l)$ in $M\Omega$, we define two mapping functions: In the FHE framework, there exists a mapping function f_{FHE} such that $(op, C_l) \xrightarrow{f_{FHE}} (FSOP_l, F_l^{FHE})$. Similarly, in the MPC framework, we have $(op, C_l) \xrightarrow{f_{MPC}} (MSOP_l, F_l^{MPC})$.
- **Dataset Generation.** We design and execute extensive protocol-computation experiments based on F_l^{MPC} and F_l^{FHE} . The dataset $\mathcal{D}^{FHE}$ for FHE is then formed as $\mathcal{D}^{FHE} = \{(FSOP_l, F_l^{FHE}, C_l^{FHE})\}$. Similarly, for the MPC framework, we have $\mathcal{D}^{MPC} = \{(MSOP_l, F_l^{MPC}, C_l^{MPC}) \mid l = 1, \dots, k\}$.
- **Regression Model Training and Selection.** For the FHE dataset $\mathcal{D}^{FHE}$, we train three regression models³. Then select the best performance model, denoted as CM_{FHE} , which is given by:

$$CM_{FHE} = \arg \min_{Model \in \{RF, XGB, LR\}} (Model, \mathcal{D}^{FHE}) \quad (2)$$

Similarly, model for MPC, denoted as CM_{MPC} , is given by:

$$CM_{MPC} = \arg \min_{Model \in \{RF, XGB, LR\}} (Model, \mathcal{D}^{MPC}) \quad (3)$$

- **Cost Prediction.** Once the optimal models CM_{FHE} and CM_{MPC} are obtained, for a given Meta-Op Ω_l : In the FHE framework, the predicted runtime $\hat{C}_l^{FHE}$ is given by $\hat{C}_l^{FHE} = CM_{FHE}(\Omega)_l$. In the MPC framework, the predicted runtime $\hat{C}_l^{MPC}$ is given by $\hat{C}_l^{MPC} = CM_{MPC}(\Omega, E)_l$ (E is the network environment).

Where F_l^{FHE} and F_l^{MPC} are the feature vectors associated with the operations in the FHE and MPC context, respectively. $MSOP_l$ and $FSOP_l$ denote the MPC-based specific protocol and FHE-based specific protocol of Meta-Op Ω_l , respectively. These features include the input-output dimensions from Ω_l , as well as the designed FHE and MPC parameters, network environment, etc. The specific implementation of CM can be found in Section IV-B.

C. Protocols Assignment

Through the cost model, we can obtain the costs of $MSOP_l$ and $FSOP_l$ for an operator Ω_l , as well as conversion costs $\hat{C}_l^{convert}$ and bootstrapping costs $\hat{C}_l^{bootstrap}$, thereby enabling protocol selection. Since this is an NP-hard problem, COSTSPHERE abstracts it as an optimization problem and uses Integer Linear Programming (ILP) for solving. Therefore, the key to the protocols assignment (PAS) lies in designing reasonable constraints to ensure that the protocols assigned by the PAS apply to different network environments while being efficient. In COSTSPHERE, PAS considers three types of constraints: ①. We need to consider the protocol conversion constraints in the hybrid assignment. During hybrid protocol allocation,

³Random Forest (RF), XGBoost (XGB), and Linear Regression (LR)

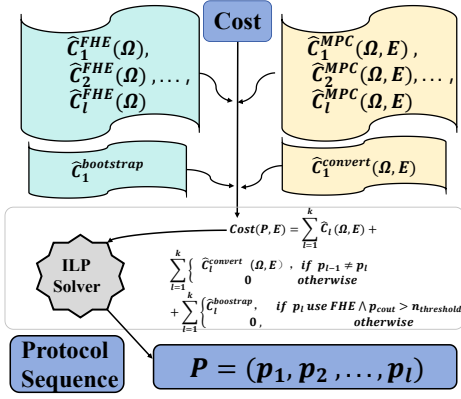


Fig. 5: protocol assignment

domain conversion between FHE and MPC involves conversion costs. Frequent switching between them will lead to high communication overhead and reduced efficiency, which must be constrained. ②. Dynamic cost penalty constraints for FHE: During the process of protocol assignment, continuous FHE multiplication requires deeper multiplication depth support, and different encryption parameters (for the same FHE operation but different multiplication depths parameters setting) lead to varying computational costs. Therefore, the FHE cost is dynamically updated with multiplication depth. We design a penalty constraint to enforce that FHE costs increase dynamically and are globally updated during the solution process based on the multiplication depth. Based on the above constraints, we established PAS, the overview is shown in Figure 5.

Given the results of the CM: $\hat{C}_1^{MPC}, \hat{C}_2^{MPC}, \dots, \hat{C}_k^{MPC}, \hat{C}_1^{FHE}, \hat{C}_2^{FHE}, \dots, \hat{C}_k^{FHE}, \hat{C}_l^{bootstrap}$, and $\hat{C}_l^{convert}$. The PAS outputs an optimal protocol assignment sequence $P = \{p_1, p_2, \dots, p_n\}$, where $p_i \in \{FHE, MPC\}$, such that the total cost of the entire computation sequence is minimized. The PS performs the following steps:

- **Cost Calculation Function Construction.** When the PS first defines a total cost function $Cost(\mathcal{O}, E)$ to measure the cost of the sequence $\mathcal{O}$ under the network environment E . And if the protocol assignments of two adjacent tasks are different, then $\hat{C}_l^{convert}$ is required. Moreover, introduce a variable $p_{count} = 0$, representing the cost of consecutive assignments of HSOP_{*l*}. If $p_{count} > n_{threshold}$, then use $\hat{C}_l^{bootstrap}$, and then reset p_{count} to 0. Thus, we have:

$$\begin{aligned}
 Cost(\mathcal{O}, E) = & \sum_{l=1}^k \hat{C}_l(\Omega, E) \quad (\text{protocol computing cost.}) \\
 & + \sum_{i=2}^{k-1} \begin{cases} \hat{C}_i^{convert}(\Omega, E), & \text{if } p_{i-1} \neq p_i \\ 0, & \text{otherwise.} \end{cases} \\
 & + \sum_{l=1}^k \begin{cases} \hat{C}_l^{bootstrap}, & \text{if } p_l = FHE \wedge p_{count} > n_{threshold} \\ 0, & \text{otherwise} \end{cases}
 \end{aligned} \tag{4}$$

- **Minimum Cost Calculation.** Using the above definitions, the optimal problem to minimize $Cost(P, E)$ is investigated in this paper. Since the optimal partitioning problem is NP-hard, we use the three designed constraints for solving the problem.

The specific process for constructing the PAS is detailed in Section IV-C.

By following these steps, construct the CM. Then, use the PAS to solve for protocol assignment, we can find the optimal combination of FHE and MPC sequences under given network environments and constraints for complex applications.

IV. COSTSPHERE SPECIFICATION

This section comprehensively presents the specific implementation details of COSTSPHERE. Including the parsing of operators in TOSA IR, the extraction of features for the cost model, the construction of the model, cost prediction, protocol assignment, and cost minimization.

A. Operator Parse

COSTSPHERE provides an interface for torch2mlir [43]. Only need to import the required model from PyTorch or build a model as needed, and then TOSA-level IR can be generated via torch2mlir. The TOSA IR contains two types of operators. One type is memory-access-intensive operators, such as *tosa.select* and *tosa.reshape*. The other type is computation-intensive operators, such as *tosa.add*, *tosa.mul*, and *tosa.matmul*.

Operator Parsing. For computation-intensive operators, we first categorize them into single-input and double-input operators. Then use the method of regular matching to obtain the names of TOSA operators, information about input and output parameters, dimensions, and the ID of the current operator, to ensure the correct computation order. Common computations like *tosa.matmul*, *tosa.add*, and *tosa.exp* are directly regarded as the Meta-Ops Ω to map the MSOP and FSOP. For memory-intensive operators like *select*, corresponding protocols need to be designed. In MPC, *select* can be directly implemented using multiplexing, i.e., the protocol $MSOP_{select} = \Pi_{if_else}(MPC)$ from SIRNN [37]. For FHE, it is implemented using homomorphic operations, with the specific principle $result = cond * x + (1 - cond) * y$, the value of *cond* can be obtained using the homomorphic symbolic function algorithms provided in Nexus [27].

Additionally, the TOSA IR includes some composite operators, such as *tosa.tanh* and *tosa.sigmoid*. For better protocol assignment, we need to decompose these composite operators. For example, the *tosa.tanh* operator is decomposed into *op.exp* operations, *op.add*, and *op.div* operations. The other is the *dense_resource* type, which usually represents neural network parameters in TOSA. Thus, COSTSPHERE provides a load interface to load model parameters locally and offers corresponding matching mappings for MPC or FHE scenarios.

Once the operators are parsed, we get a Meta-op. This Meta-op holds information about the operators, including the current operator ID, operator name, input and output parameter

Protocol MPC- $\Pi_{\text{exp}}^{m,s,n,s'}$

Input: $\mathcal{A}$ & $\mathcal{B}$ hold share $\langle x \rangle^m$
Output: $\mathcal{A}$ & $\mathcal{B}$ get $\langle y \rangle^n$ for $y = \exp(-x)$
Execution:

- Each party calls $\Pi_{\text{DigDec}}^{m,d}$ to get $\langle h_i \rangle_{i \in [k]}^d$, $h_i \in 0, 1^d$ s.t. $x = h_{k-1} || \dots || h_0$, ($dk = m$).
- For $i \in [k]$, let $\langle L_i \rangle : 0, 1^d \rightarrow \mathbb{Z}_2^{s'+2}$ s.t. $\langle L_i(j) \rangle = \text{FIX}^a(\Pi_{\text{LUT}}^{T,m,n}(2^{di-s}j), s'+2, s')$
- Each party gets $\langle g \rangle$ by calling $\Pi_{\text{SMultTR}}^{s'+2, s'+2, 2s'+2, s}(\langle L_{k-1}[x_{k-1}] \rangle * \dots * \langle L_0[x_0] \rangle)$. $\langle g \rangle$ has bitwidth $s' + 2$ and scale s' .
- Each party return $\langle y \rangle^n = \Pi_{\text{SExt}}(\langle g \rangle, s' + 2, n)$

^a"FIX" indicates the process of converting the result obtained from the lookup table into a fixed-point number and then transforming it into a Ring element.

Fig. 6: Protocol MPC- Π_{exp} for 2PC.

Protocol FHE- Π_{exp}

Input: $\mathcal{B}$ holds FHE-ciphertext $\llbracket \mathbf{x} \rrbracket_{\mathcal{A}}$
Output: $\mathcal{B}$ gets $\llbracket y \rrbracket_{\mathcal{A}}$ for $y = \exp(-x)$
Execution:

- Encode $\frac{1}{2^r}$ through SIMD, obtaining $p = \text{Plaintext}(\frac{1}{2^r})$. Calculate $\llbracket l \rrbracket_{\mathcal{A}} = H\text{Mul_Plain}(\llbracket \mathbf{x} \rrbracket_{\mathcal{A}}, p)$, and perform $H\text{Rescale}(\llbracket l \rrbracket_{\mathcal{A}})$.
- Encode 1 through SIMD, getting $n = \text{Plaintext}(1)$. Compute $\llbracket y \rrbracket_{\mathcal{A}} = H\text{Add_Plain}(\llbracket \mathbf{x} \rrbracket_{\mathcal{A}}, n)$.
- for— i , where $i < \log(2^r)$ do
 - $H\text{Square}(\llbracket y \rrbracket_{\mathcal{A}}, \llbracket y \rrbracket_{\mathcal{A}})$
 - $H\text{Relin}(\llbracket y \rrbracket_{\mathcal{A}})$
 - $H\text{Rescale}(\llbracket y \rrbracket_{\mathcal{A}})$

Fig. 7: Protocol FHE- Π_{exp} for 2PC.

IDs, and the dimensions of input and output parameters. And predefined costs such as HE-cost, SS-cost, etc. Since we can determine whether a plaintext-ciphertext computation or a ciphertext-ciphertext computation is needed by checking if the input/output parameters are constants. The predefined costs can be obtained through the CM, and protocol assignments are then handled by the PAS.

It should be noted that COSTSPHERE focuses on privacy inference in the 2-PC scenario. Therefore, when the TOSA-Ops parses to Meta-Ops, we provide the protocol interfaces for Meta-Op under the 2-PC scenario. For example, when *tosa.exp* is mapped to Ω_{exp} , we provide the protocol MPC- $\Pi_{\text{exp}}^{m,s,n,s'}$ [37]. The specific protocol of them under MPC is as Figure 6. Where m and n denote bitlength, s and s' denote scale. For the implementation details of specific protocols such as Π_{LUT} and Π_{SExt} , please refer to SIRNN [37]. For the protocol of Ω_{exp} implemented under FHE, we employ the approximation approach that is customarily used in [25], [27]. The Taylor series approximation is as follows:

$$\text{op.exp} \approx (1 + \frac{x}{2^r})^{2^r}, \quad x \leq 0 \quad (5)$$

When $r = 8$, the average error is less than 10^{-5} . Then, the specific protocol for Ω_{exp} based on FHE is as shown in Figure 7.

TABLE IV: Basic operators for CM.

FHE operators		MPC operators	
add_plain	encode/decode	add	great_equal
add/sub	encrypt/decrypt	sub	if_else
sub_plain	rotate_vector_inplace	abs	reciprocal
mul_plain	mod_switch_to_next	exp	mul_bots_private
mul	relinearize_inplace	rsqrt	mul_first_public
square	rescale_to_next_inplace	mul_second_public	He_ss_convert
select	negate_inplace		

The implementation of the MPC protocols, COSTSPHERE employs the protocols provided by SIRNN [37] lib and SCI lib [26]. For protocols implemented with FHE, COSTSPHERE uses the Π_{sign} protocols provided by Nexus [27]. The FHE- Π_{matmul} from FASTLMPI [40] and BOLT [24]. It should be noted that we focus on the adaptation to the network environment, and the performance issues of a single protocol are not within the scope of COSTSPHERE's concern.

Since the protocols are based on Meta-Ops mapping, they are highly independent, facilitating convenient replacement, upgrade, or performance update of individual protocols. Thus, when there is a new protocol for replacement, only the single protocol needs to be modified, and the operator's CM needs to be readjusted without the need to modify the entire CM.

B. Cost Model Design

This section outlines the implementation of the Cost Model (CM) for the FHE and MPC protocols. FHE computations, which are communication-free, have costs driven by encryption parameters and input/output data dimensions. Conversely, MPC requires extensive communication, with costs influenced by input/output dimensions, network bandwidth, latency, and ring-based computations using fixed-point number encoding, which necessitates consideration of bit length and scale. Due to these distinct characteristics, we develop separate cost models for FHE and MPC to accurately reflect their respective computational and communication overheads.

As shown in Figure 4, the specific steps of the CM are as follows:

Operator-protocol matching and information extraction. For the operator sequence $M\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_k\}$, first map it to FSOP and MSOP. For FSOP, FHE only includes the operators shown in Table IV (left), and we conduct the cost model CM_{FHE} by taking these operators. The advantage of this approach is that the established cost model can be applied to the cost analysis of any FHE application. When implementing the FSOP Π_{exp} , as shown in Figure 7, the cost of Π_{exp} can be obtained by simply calling the basic operators. For MSOP, shown in Table IV (right), it is directly mapped to Meta-Op. After the operator and the protocol are matched, we can obtain feature information from them, including the calculation dimensions.

Feature creation. After obtaining the input and output dimensions, *input_dim* and *output_dim*, of the protocol from Meta-Ops. In addition, for FSOP, add the encryption parameters: *poly_modulus_degree* and *coeff_modulus* as additional features. Moreover, in the FHE framework,

based on the $input_dim$ and $poly_modulus_degree$, we can supplement the feature $cipher_size = \frac{input_dim}{poly_modulus_degree/2}$. For MSOP, we add $bitlength$, $scale$, $bandwidth$ and $net_latency$ as additional features. Thus we have $\mathbf{F}_l^{FHE} = \{input_dim, output_dim, poly_modulus_degree, coeff_modulus, cipher_size\}$ and $\mathbf{F}_l^{MPC} = \{input_dim, output_dim, bitlength, scale, bandwidth, net_latency\}$.

Training data generation. Once the features $\mathbf{F}_l^{FHE}$ and $\mathbf{F}_l^{MPC}$ are confirmed, we can generate the dataset based on these features. Specifically: For $FSOP_l$, the values of $input_dim$ and $output_dim$ increase from 10 to 34000 with a step size of 300. The $poly_modulus_degree$ increases from 2^{10} to 2^{15} with a step size of 2^1 . The $coeff_modulus$, which represents the supported computational depth, increases from 4 to 21 with a step size of 1. Under these settings, we conduct actual runs of specific FSOP operations to obtain the runtime C_l^{FHE} under different feature combinations. For $MSOP_l$, the values of $input_dim$ and $output_dim$ increase from 10 to 34000 with a step size of 200. Here, the $bitlength$ is fixed at 37, and the $scale$ is fixed at 13. The $bandwidth$ increases from 100 to 1000 with a step size of 100, and the $net_latency$ is selected from the set $\{3, 5, 10, 20, 30, 40\}$. The $bandwidth$ and $net_latency$ are simulated using the Linux traffic control tool *tc*. Under these settings, we conduct actual runs of specific MSOP operations to obtain the runtime C_l^{MPC} . At this point, the datasets $\mathcal{D}^{FHE} = \{(FSOP_l, \mathbf{F}_l^{FHE}, C_l^{FHE})\}$ and $\mathcal{D}^{MPC} = \{(MSOP_l, \mathbf{F}_l^{MPC}, C_l^{MPC})\}$ can be obtained.

Cost model construction. Three regression models, namely Random Forest, XGBoost, and Linear Regression, are employed for cost prediction. However, we found that directly training with the datasets $\mathcal{D}^{FHE}$ and $\mathcal{D}^{MPC}$ cannot yield accurate prediction results. The reason is that during training, we usually need to convert FSOP or MSOP into one-hot encoding. In fact, for both FSOP and MSOP, the actual running time of each protocol varies significantly under given feature settings. For example, the actual running time of the *add_plain* ranges from $7.25 \mu s$ to $7914.75 \mu s$, while that of *encode* ranges from $170.25 \mu s$ to $29515.875 \mu s$. This disparity affects the model's learning ability and leads to large errors. Therefore, we adopt the Multi-Output Model approach [47] to segment the data according to the protocols. For instance, the data from $1 \sim 2000$ corresponds to the *add_plain*, and the data from $2001 \sim 4000$ corresponds to the *sub_plain*. Meanwhile, each segment of data is input into the corresponding three regression models for training, and the model CM_{FHE} (2) and CM_{MPC} (3) with the best performance among the three is selected as the cost model for the current protocol. This decoupled setting among the models of different protocols not only solves the problem of mutual influence caused by the large differences in the actual running times of different protocols but also facilitates protocol updates. When a protocol is upgraded, only its corresponding cost model needs to be replaced. In this way, we can obtain the cost models for FSOP and MSOP, respectively.

Cost prediction. We can provide the Meta-Ops, the FHE encryption parameters such as $poly_degree$, as well as network

bandwidth and latency, COSTSPHERE can predict the cost runtime $\hat{C}_l$ of the specific FSOP and MSOP by using the cost model CM_{FHE} and CM_{MPC} , respectively.

C. Hybrid protocols assignment

Through PAS, the optimal protocol is assigned to each operation, introducing the designed constraints to ensure that the final operation sequence achieves the optimal protocol combination under specific network conditions. The specific implementation steps of PAS are as follows.

Constraint Construction. Based on the analysis of the cost of computation, we define the following decision variables to ensure that for each operator's protocol assignment, only one cost is selected between MSOP and FSOP. We also introduce conversion cost and bootstrapping cost decision variables to constrain FHE and MPC domain conversions.

- $x_l^{HE} \in \{0, 1\}$: If a operator Ω_l is executed with $\Pi_{\Omega_l}^{HE}$.
- $x_l^{MPC} \in \{0, 1\}$: If a operator Ω_l is executed with $\Pi_{\Omega_l}^{MPC}$.
- $s_l^{HE \rightarrow MPC} \in \{0, 1\}$: If a protocol Π switch from $\Pi_{\Omega_l}^{HE}$ to $\Pi_{\Omega_l}^{MPC}$. at operator Ω_l ($l \geq 2$).
- $s_l^{MPC \rightarrow HE} \in \{0, 1\}$: If a protocol Π switch from $\Pi_{\Omega_l}^{MPC}$ to $\Pi_{\Omega_l}^{HE}$. at operator Ω_l ($l \geq 2$).
- $bot_l \in \{0, 1\}$: If bootstrapping is required at Ω_l when $\Pi_{\Omega_l}^{HE}$ continuously uses more than the threshold value $n_{threshold}$.

The definition of the objective function. Based on the decision variables and the cost calculation function as shown in equation 4, we construct the objective function.

$$\begin{aligned}
 Object(\mathcal{O}, E) = & \underbrace{\sum_{l=1}^k \left(x_l^{HE} \cdot \hat{C}_{HE}(\Omega_l) + x_l^{MPC} \cdot \hat{C}_{MPC}(\Omega_l, E) \right)}_{\text{protocol computation cost}} \\
 & + \underbrace{\sum_{l=2}^k \left(s_l^{HE \rightarrow MPC} \cdot \hat{C}^{convert}(\Omega_l, E) + s_l^{MPC \rightarrow HE} \cdot \hat{C}^{convert}(\Omega_l, E) \right)}_{\text{protocol switching cost}} \\
 & + \underbrace{\sum_{l=1}^k bot_l \cdot \hat{C}^{bootstrap}(\Omega_l)}_{\text{bootstrapping cost}}.
 \end{aligned}$$

The definition of constraint conditions. To ensure the effectiveness of protocol assignment, it is necessary to construct constraint conditions for the solution model to limit the feasible solution space and ensure that the protocol assignment meets the requirements. The constraints are as follows.

- Unique protocol assignment:

$$x_l^{HE} + x_l^{MPC} = 1, \quad \forall l \in [1, k] \quad (6)$$

- Protocol switching indicators:

$$s_l^{HE \rightarrow MPC} \geq x_{l-1}^{HE} + x_l^{MPC} - 1, \quad \forall l \in [2, k] \quad (7)$$

$$s_l^{HE \rightarrow MPC} \leq x_{l-1}^{HE}, \quad \forall l \in [2, k] \quad (8)$$

$$s_l^{HE \rightarrow MPC} \leq x_l^{MPC}, \quad \forall l \in [2, k] \quad (9)$$

$$s_l^{MPC \rightarrow HE} \geq x_{l-1}^{MPC} + x_l^{HE} - 1, \quad \forall l \in [2, k] \quad (10)$$

$$s_l^{MPC \rightarrow HE} \leq x_{l-1}^{MPC}, \quad \forall l \in [2, k] \quad (11)$$

$$s_l^{MPC \rightarrow HE} \leq x_l^{HE}, \quad \forall l \in [2, k] \quad (12)$$

- Bootstrapping constraint: Let $n_{\text{threshold}}$. For any window of length $n_{\text{threshold}}$, bootstrapping occurs at the end of the window if all $n_{\text{threshold}}$ preceding operators use $\Pi_{\Omega_l}^{HE}$:

$$b_l \geq \frac{1}{n_{\text{threshold}}} \sum_{j=l-n_{\text{threshold}}+1}^l x_j^{HE} - (n_{\text{threshold}} - 1), \quad \forall l \in [n_{\text{threshold}}, l] \quad (13)$$

- Dynamic update of HE costs:

$$x_l^{HE} \cdot \hat{C}_{HE}(\Omega_l) = f(x_l^{HE} \cdot \hat{C}_{HE}(\Omega_l) \cdot a + b) \quad (14)$$

When $x_l^{HE} = 1$ consecutively, we calculate $\text{count}(x_l^{HE}) = \text{count}(x_l^{HE}) + 1$. As the $\text{count}(x_l^{HE})$ increases, it indicates an increase in HE multiplication depth. Based on this condition, a penalty factor a and a bias term b can be fitted.

- Parameter Type Determination: If the type of an operator's input parameter is a constant, then during homomorphic operations, we opt for homomorphic ciphertext-plaintext computation. Otherwise, we select homomorphic ciphertext-ciphertext computation.

Initiate ILP solving. By following these steps, we can invoke the ILP [48] solver to obtain the protocol assignment that results in the lowest cost in a specific environment.

- Extract the protocol assignment for each operator: assign Ω_l to $\Pi_{\Omega_l}^{HE}$ if $x_l^{HE} = 1$, otherwise to $\Pi_{\Omega_l}^{MPC}$.
- Record the positions of protocol switches and any bootstrapping events.
- Report the minimized total cost and the full protocol schedule.

When an operator's input includes a constant, we set the $\hat{C}_{HE}$ cost to the FHE ciphertext-plaintext computation cost. The ILP model ensures each operator is assigned exactly one protocol, switching costs occur only when the protocol changes, and bootstrapping costs are applied when HE is used for more than $n_{\text{threshold}}$ consecutive operators. Finally, PAS provides a globally optimal protocol schedule that minimizes the total cost.

V. EVALUATION

In this section, we evaluate the performance of COSTSPHERE in three aspects: the prediction performance of the CM III-B, the PAS III-C performance based on dynamic constraints, and the inference performance of COSTSPHERE for allocating protocols for the BERT model.

A. Experimental Setup

COSTSPHERE is implemented in C++ and we use EMP toolkit⁴ [49] to implement communication between the parties. Leveraging the SCI⁵ library in EzPC [50] for fixed-point numerical computations, and the MPC protocol is derived from SIRNN [37] and BOLT [24]. We use the changed SEAL⁶ [51] library for RNS-CKKS homomorphic encryption from Nexus [27]. The external libraries are compiled by GCC (version 11.4.0) on Ubuntu 20.04.

Benchmark Environment: All the following experiments are performed on a rack server with the Intel(R) Xeon(R) Gold 6238R CPU @ 2.20GHz and 377 gigabytes of RAM. We control the communication bandwidth using the Linux Traffic Control (tc) command. We simulate the WAN environments, which are LAN = {2Gbps, 0ms}, WAN = {100Mbps, 80 ms}. And the number of threads is set to 32. The SoftMax function has a relatively small dimension, so its protocol is implemented using a single thread.

Concrete FHE's and MPC's Parameters: This paper uses bit-length $k = 37$ and precision $s = 12$ for the expressed of fixed-point numbers, and use the SEAL parameters $\text{HE}_{\text{SCI}} = \{N = 8192, q \approx 180, p \approx 29\}$ (Similar to [24], [52]) for FHE.

Model Architectures: We measure the performance of COSTSPHERE on BERT_{BASE} [22] and denote the number of layers (i.e., Transformer blocks) as L , the hidden size as H , and the number of self-attention heads as A . The size of feed-forward is $4H$, i.e., 3072 for the $H = 768$ and $L = 12$.

B. Performance of Cost Model

The CM provides high extensibility and replaceability due to its decoupled FHE and MPC cost models, with each operator's cost prediction model being independent. Some basic operators in COSTSPHERE are shown in Table IV. We demonstrate individual operators' performance during cost prediction by randomly selecting 200 values from the test set for error analysis. Figures 8 and 9 show the results. The green line (error) is nearly a smooth straight line, with only a few jumps in some values. We analyze these fluctuations and conclude that they might be due to program execution instability, influenced by cache effects. And we apply median filtering to address values with significant fluctuations.

We also calculate the fitting capabilities of three different regression models during the model training process. Table V presents their Mean Absolute Error (MAE) and Coefficient of Determination (R^2) values.

We analyze the results in Table V, where the values of R^2 all exceeded 0.95 to 0.99. This indicates that the regression model fits the data quite well. However, the MAE values of different protocols vary significantly. The fundamental reason for this is that the costs of different protocols are quite different. For instance, the computational complexity of "add" and "if_else" under MPC is different. In addition, we observe that the MAE

⁴<https://github.com/emp-toolkit/emp-tool>

⁵<https://github.com/mpc-msri/EzPC/tree/master/SCI>

⁶<https://github.com/microsoft/SEAL>

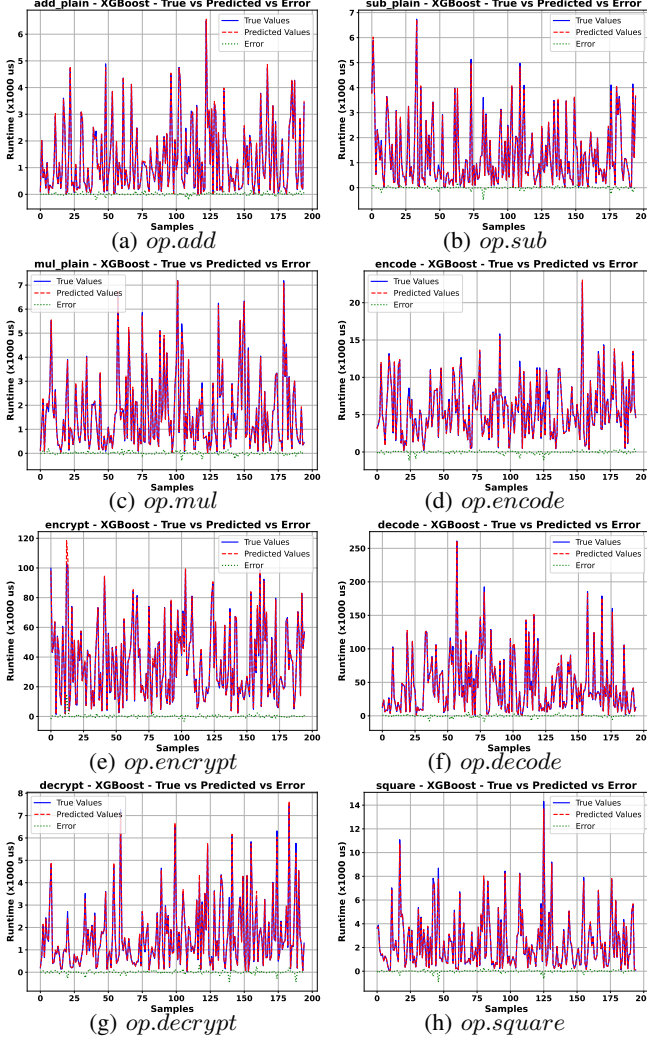


Fig. 8: (FHE) Cost model performance of the computationally-intensive operators.

TABLE V: Comparison of the performance of the cost model in the agreement. (We list a few of the commonly used protocols.)

Model name	FSOP	MAE(ms)	R ²	MSOP	MAE(μ s)	R ²
RandomForest	select	4.972	0.997	if_else	52.71	0.959
XGBoost		2.675	0.999		69.07	0.957
Polynomial		63.8	0.823		827.84	0.159
RandomForest	add	149.6	0.984	add	3.42	0.963
XGBoost		47.11	0.998		1.21	0.996
Polynomial		534.50	0.825		2.449	0.983
RandomForest	sub	99.22	0.983	sub	4.610	0.968
XGBoost		29.26	0.998		2.491	0.994
Polynomial		431.52	0.806		3.309	0.985
RandomForest	negate	90.488	0.967	abs	25.709	0.978
XGBoost		40.500	0.992		18.509	0.997
Polynomial		274.05	0.841		182.437	0.807
RandomForest	mul	125.334	0.984	mul	30.81	0.993
XGBoost		37.427	0.999		23.907	0.996
Polynomial		506.607	0.831		225.923	0.784

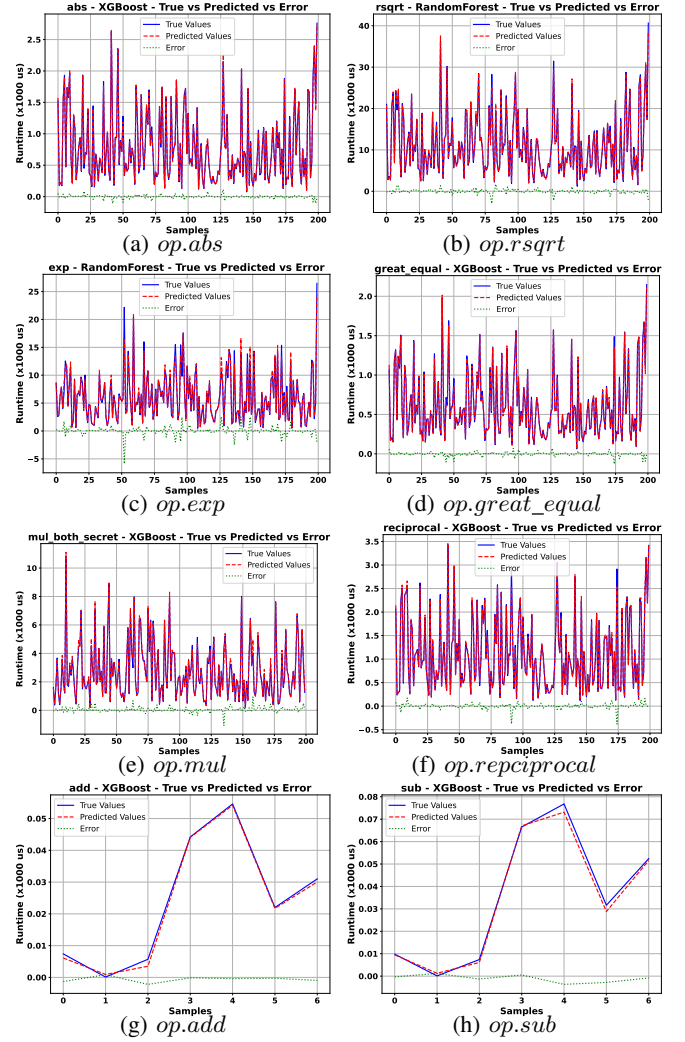


Fig. 9: (MPC) Cost model performance of the computationally-intensive operators.

of some protocols is relatively large. This is because during the experiment, the distribution range of our training data is wide, such as the input dimensions ranging from 10 to 34,000. The Protocol costs are easily influenced by the computation dimensions, which affect the MAE of individual operators. Therefore, we trained three regression models and selected the one with the smallest MAE and the highest R^2 as the cost model.

C. Performance of Protocol Assignment (PAS)

We verify the performance of the protocol assignment using the PAS. and compare the hybrid protocols with protocols that only use MPC or FHE technologies. As shown in Table VI, we present the protocol assignment performance of the SoftMax, GeLU, and LayerNorm operators used in the BERT model. As a result, for the hybrid protocol assigned by PAS, we test its actual running time and compare it with the predicted time. The difference is small, such as for the SoftMax, LayerNorm, and GeLU functions. The differences between the predicted results and actual time are 27.6 ms,

TABLE VI: Performance analysis of the PAS. The input of PAS is Meta-Ops, and the computing cost is obtained through the CM. The output is the hybrid protocol and the predicted runtime ("Pred. Runtime"). "Runtime" refers to the actual running time of the hybrid protocol. (Under LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms} settings.)

Operation	Protocol Assignment												Runtime	Pred. Runtime
SoftMax_LAN	reduce_max	sub	exp	reduce_sum	reciprocal	mul							419.0 ms	391.4 ms
SoftMax_WAN	reduce_max	sub	exp	reduce_sum	reciprocal	mul							1771.89 ms	1880.0 ms
LayerNorm [‡]	cast	reduce_sum	mul	sub	mul	reduce_sum	mul	cast	reduce_sum	mul	add	rsqrt	955.2 ms	1019.4 ms
	sub	mul	reshape	mul	reshape	add								
GeLU_LAN	mul	abs	mul	add	mul	mul	add	mul	mul	add	mul	mul	3.996 s	3.255 s
	add	reciprocal	mul	mul	sub	greater_equal	negate	select	add	mul	mul	mul		
GeLU_WAN	mul	abs	mul	add	mul	mul	add	mul	mul	add	mul	mul	105.22 s	92.17 s
	add	reciprocal	mul	mul	sub	greater_equal	negate	select	add	mul	mul	mul		

[‡] In the LAN and WAN settings, LayerNorm assigned the same hybrid protocol.
Note: ● denotes FSOP, ○ denotes MSOP, ◐ denotes FSOP_plain.

64.2 ms, and 0.741 s under LAN settings, respectively. From Table VI, it can be seen that as the network setting changes from LAN to WAN, the protocol assignment is more inclined to choose FSOP.

TABLE VII: Performance comparison between hybrid protocols and protocols using only FHE or MPC. (Under LAN={2 Gbps, 0 ms} setting.)

Protocols	Predicted Runtime (s)	Hybrid Runtime (s)	MPC Runtime (s)	FHE Runtime (s)
SoftMax	0.391	0.419	12.074	1.840
GELU	3.255	3.996	6.603	56.143
LayerNorm	1.019	0.955	13.779	36.487

To demonstrate the advantages of the hybrid protocol, we compare it with the protocols that solely used MPC technology or FHE technology. As Table VII shows, for the SoftMax function, the performance of the hybrid protocol was 4.39 times better than that of the FSOP. Compared with MSOP, the performance has been improved by 28.8 times, while the communication overhead has been reduced by 7 GB (MPC-based=7913.86 MB, hybrid-based=5.303). For GeLU, the performance of the hybrid protocol is 16.8 times higher than that of the FSOP and 1.7 times higher than that of the MSOP. At the same time, the communication overhead is reduced by 8.34 GB (MPC-based=10337.99 MB, hybrid-based=1798.23). For the LayerNorm function, the hybrid protocol outperforms the FSOP by 38.2 times, and outperforms the MSOP by 14.4 times, while reducing the communication volume by 0.96 GB (MPC-based=1001.81 MB, hybrid-based=12.79 MB). For the special case of the SoftMax function, as shown by the gray numbers in Table VII, its MSOP is slower than the FSOP in the LAN environment. We split the protocol and found that using MPC technology to implement the exp protocol takes 11.8 seconds, accounting for 97.8% of the all execution time. Therefore, an efficient MPC-based exp protocol is urgently needed.

We also verify the adaptability of the network environment. We set the network environment to WAN = {100 Mbps, 80 ms}, and adopt PAS for protocol allocation. We obtain a hybrid protocol and compare it with the hybrid protocol generated under the LAN environment. The results are shown

TABLE VIII: The adaptability of the hybrid protocol to the network environment.(LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms})

Operation		Runtime (s)		Comm. (MB)
		LAN	WAN	
SoftMax_LAN	128 × 128	0.419	12.07	0.442
SoftMax_WAN		1.88	5.26	0
GeLU_LAN	128 × 3072	3.996	180.57	1798.23
GeLU_WAN		18.32	105.22	884.20
LayerNorm	128 × 768	0.96	3.32	12.79

in Table VIII. We use COSTSPHERE to assign protocols in both LAN and WAN environments, obtaining various protocol combinations to adapt to different network conditions. For matrix multiplication (Mat_LAN, Mat_WAN), in the LAN environment, the communication cost of Mat_LAN is 6.87 MB, but its execution time is 8× as fast as that of Mat_WAN. This indicates that when the network environment is well, it is reasonable to reduce the computational load by introducing an acceptable amount of communication. In the WAN environment, the Mat_WAN communication overhead is 2.35, and its execution time is 4.12× faster than that of Mat_LAN. Similarly, both GeLU and SoftMax demonstrate the network environment adaptation feature of COSTSPHERE. For LayerNorm, since the use of reduce_sum with FHE is very time-consuming, and considering the dimension of LayerNorm is 128 × 768, during the summation process, it is necessary to rotate a ciphertext $\log_2(768)$ times. However, we set $poly_degree = 8192$, thus requiring 24 FHE ciphertexts. Therefore, when the network environment is a WAN, using MPC to implement reduce_sum is more efficient, because it only requires transposing the shared data and then performing summation.

D. Private BERT inference performance

Microbenchmarks: We test the hybrid protocol of SoftMax, GeLU, and LayerNorm generated by COSTSPHERE, and evaluate its performance in both LAN and WAN environments. We also compare the results with those of the current SOTA models, IRON [53], BOLT [24], Nexus [27], and BumbleBee [25]. The experimental results are shown in Table IX.

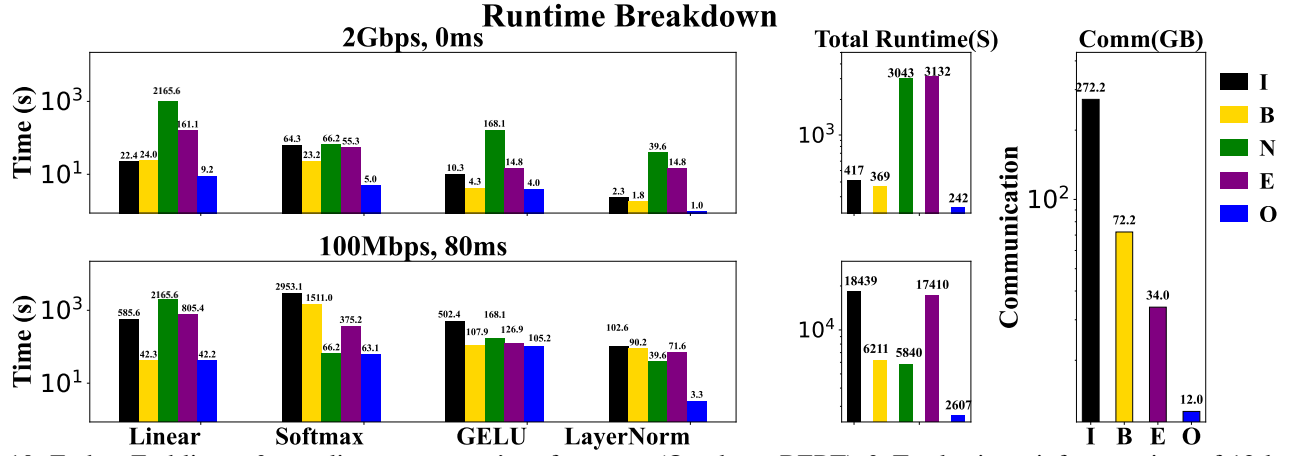


Fig. 10: End-to-End linear & non-linear operators’ performance (One layer-BERT) & Total private inference time of 12-layers BERT & Total communication overhead of 12-layers BERT (Nexus for 1-layer BERT). (“I” denotes IRON, “B” denotes BOLT, “N” denotes Nexus, “E” denotes BumbleBee, “O” denotes COSTSPHERE. Under LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms} network environment settings. “Linear” represents all the matrix multiplication in BERT. Refer to Table XI.

TABLE IX: Comparison of Hybrid protocols with SOTA in terms of running time and communication costs. Timing results are averaged from 5 runs. Under LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms} network environment settings.

Method		Time (s)		Comm.(MB)
		LAN	WAN	LAN — WAN
SoftMax 128 × 128	IRON	5.36	246.09	3356.92
	BOLT	1.96	125.92	1285.13
	Nexus	5.51	5.51	-
	BumbleBee	4.61	31.29	85.96
	COSTSPHERE	0.42	5.26	0.442 — 0
LayerNorm 128 × 768	IRON	2.33	102.60	871.46
	BOLT	1.82	90.17	844.6
	Nexus	39.60	39.60	-
	BumbleBee	14.85	71.67	98.14
	COSTSPHERE	0.96	3.32	12.79
GeLU 128 × 3072	IRON	10.30	502.39	7524
	BOLT	4.33	107.89	2769.89
	Nexus	168.13	168.13	-
	BumbleBee	14.85	130.09	601.79
	COSTSPHERE	3.996	105.22	1798.23—884.20

The hybrid protocol generated by COSTSPHERE performs well both in LAN and WAN environments. Such as SoftMax, the protocol assigned by COSTSPHERE achieves a 4-fold improvement over the SOTA BOLT [24] in the LAN environment. In the WAN environment, due to the sole adoption of FHE technology, it is slightly faster than Nexus [27]. Compared with the SOTA Bumblebee [25], the communication overhead is reduced by 85.52 MB. The hybrid protocol of LayerNorm requires only 0.96 seconds in a LAN environment and 3.32 seconds under WAN. The communication overhead is 13% of that of BumbleBee. Due to the use of polynomial approximation methods by BOLT, Bumblebee, etc. for GeLU, which is more efficient, the calculation time of the GeLU hybrid protocol in COSTSPHERE has only slightly increased.

End-to-End private inference performance of BERT. In Table X, we compare the performance of the hybrid protocol generated by COSTSPHERE with other private inference methods of BERT, including IRON [53], BOLT [24], BumbleBee

[25], and Nexus [27]. In summary, under different network environments, COSTSPHERE achieves $6.68 \times -12.92 \times$ improvements in inference runtime compared to BumbleBee (with BumbleBee runtime of 3131.64, 17409 vs. COSTSPHERE 242.4, 2606.76 under LAN and WAN.) We also provide Figure 10 to visually demonstrate the performance of the hybrid protocols generated by COSTSPHERE.

TABLE X: End-to-end (12-layer Transformer encoder-only model (BERT)) comparisons with existing private inference frameworks. Under LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms} network environment settings. Framework marked with † is FHE-based (1-layer BERT).

Method	Time (s)		Comm.(GB)	
	LAN	WAN ₄	LAN	WAN ₄
Nexus [27]†	3043.14	5840.34	0.16	0.16
BumbleBee [25]	3131.64	17409	34.03	34.03
IRON [53]	417.08	18439	272.22	272.22
BOLT [24]	369.19	6211.3	72.21	72.21
COSTSPHERE	242.4	2606.76	50.01 (GB)	15.44 (GB)

Performance breakdown of COSTSPHERE (BERT). Table XI breaks down the COSTSPHERE inference runtime and communication for BERT. The input to the model consists of 128 tokens. Under the LAN setting, the GeLU function and the first matrix multiplication make up about 76% of the communication costs. In the WAN setting, the activation function is the most expensive operation, which takes 81% of the total inference time. This indicates that the bottleneck of private BERT inference lies in the nonlinear functions.

VI. RELATED WORK

Cost model for privacy computing. Several studies focus on the cost models for privacy computing tasks, such as [29], [31], [28], [17]. When CostCo [17] developed a framework for automatically generating cost models at the circuit level based on design, they used the number of gates and the depth of the circuit as features to design the cost function and then solved it. CheapMPC [28] represents the program as a

TABLE XI: Performance breakdown of (BERT_{BASE}). Under LAN={2 Gbps, 0 ms}, WAN={100 Mbps, 80 ms} network environment settings.

Operation		Runtime (s)		Comm. (MB)	
		LAN	WAN	LAN	WAN
MatrixMul	$(\mathbb{Q}^{d_{128 \times 768 \times 768}}) \times 3$	1.76	7.50	1479.34	84.72
MatrixMul	$(\mathbb{Q}^{d_{128 \times 64 \times 128}}) \times 12$	0.61	8.26	322.96	0
SoftMax	$(\mathbb{Q}^{d_{128 \times 128}}) \times 12$	5.05	63.12	5.30	0
MatrixMul	$(\mathbb{Q}^{d_{128 \times 128 \times 64}}) \times 12$	0.92	7.90	322.98	26.85
MatrixMul	$\mathbb{Q}^{d_{128 \times 768 \times 768}}$	0.87	3.91	54.18	28.24
LayerNorm	$\mathbb{Q}^{d_{128 \times 768}}$	0.96	3.32	12.79	12.79
MatrixMul	$\mathbb{Q}^{d_{128 \times 768 \times 3072}}$	2.65	8.19	107.29	107.29
GeLU	$\mathbb{Q}^{d_{128 \times 3072}}$	4.00	105.22	1798.23	884.20
MatrixMul	$\mathbb{Q}^{d_{128 \times 3072 \times 768}}$	2.42	6.49	161	161
LayerNorm	$\mathbb{Q}^{d_{128 \times 768}}$	0.96	3.32	12.79	12.79
Total		20.2	217.23	4276.86	1317.88
Total(12-layers)		242.4	2606.76	50.01 (GB)	15.44 (GB)

DAG (Directed Acyclic Graph), records the execution time and communication time of each operator, and sums them up to serve as the cost function for establishing the cost model. [31] represents eight commonly used basic operations, and it solves them using the cost function in [29]. These costs are achieved by allocating importance parameters and time accumulation.

Hybrid protocol assignment. A hybrid protocol can achieve higher performance. Existing works, such as [31], [30], [28], and so on, focus on the assignment of the optimal protocol combination. [31] used both heuristic methods and integer programming to solve the problem, and analyzed the performance differences between the two approaches. CheapMPC [28] employs two heuristic algorithms for the solution, namely Bottom-up Heuristic and Top-Down Heuristic. Silph [30] adopts the graph partitioning method and designs a hybrid ILP optimization framework to assign the protocol.

Hybrid Protocol framework. Frameworks like ABY [15], ABY3 [16], EzPC [50], and Secretflow [54], through the design of DSLs to build dedicated compilers, convert complex applications into computations supported by the underlying cryptographic libraries. ABY and ABY3 provide hybrid protocol frameworks that support arithmetic sharing, boolean sharing, and Yao’s sharing. EzPC has designed an end-to-end compiler that converts applications into EzPC program descriptions and then invokes the protocols of the underlying SCI library to implement privacy computing tasks. Secretflow achieves this by designing an MLIR dialect to match the application with the underlying cryptographic library, thereby enabling automated translation and selecting a fixed hybrid protocol to execute privacy computing tasks.

VII. DISCUSSION

Extensibility of COSTSPHERE. COSTSPHERE constructs independent cost models for each matched protocol based on the operator granularity of TOSA IR, covering all Meta-Ops in BERT to perform private inference for the BERT model. It can be extended to models like GPT [23], DeepSeek-R1 [55] based on MOE Transformer [40]. However, support for visual models is limited as the underlying protocol only provides a generic matrix multiplication protocol and lacks specialized optimization for convolution operations. This limitation stems from the maturity of the underlying protocol. Nevertheless,

COSTSPHERE supports the extension of cost models for basic operators beyond those in Table 3, enabling support for more Torch machine learning models.

Limitations of COSTSPHERE. COSTSPHERE can provide optimal protocol assignment in different network environments through its cost model and dynamic ILP solver. However, the current network environments are defined based on predefined bandwidth and latency, whereas in real-world applications, network conditions may fluctuate in real-time, which poses a challenge for COSTSPHERE. Future improvements could integrate real-time network monitoring services to dynamically adjust the ILP solver, enabling finer-grained adaptation to evolving network conditions and further enhancing performance.

VIII. CONCLUSION & FUTURE WORK

This paper proposes a cost-model-driven hybrid protocol assignment framework for 2-PC neural network inference. This method generates protocol assignments for different network environments, addressing the limitations of existing methods in adapting to network context fluctuations. Through experiments on the representative BERT model, which involves larger matrix sizes and more complex activation functions, the hybrid protocol assigned by COSTSPHERE demonstrates significant performance improvements in private-preserving inference tasks. Future research will focus on two primary directions to enhance COSTSPHERE. Convolution operations currently use generic matrix multiplication protocols, which are not ideal for vision tasks. Enhancing these protocols will boost inference efficiency in privacy-preserving vision applications. COSTSPHERE works under fixed network conditions, reducing its effectiveness during real-time fluctuations. A dynamic protocol strategy that adapts to real-time changes will improve robustness in dynamic settings.

REFERENCES

- [1] C. Dong, J. Weng, J. Liu, Y. Zhang, Y. Tong, A. Yang, Y. Cheng, and S. Hu, “Fusion: Efficient and secure inference resilient to malicious servers,” 2023.
- [2] W. Liu, P. Lu, and J. Gao, “Rgcgt: A high-order feature learning framework for predicting disease-metabolite interaction using residual graph convolution and graph transformer,” *Expert Systems with Applications*, p. 126637, 2025.
- [3] J. Xu and Y. Wang, “Enhancing healthcare recommendation systems with multimodal llms-based moe architecture,” in *5th International Conference on Signal Processing and Machine Learning (CONF SPML 2025)*, vol. 2025. IET, 2025, pp. 123–129.
- [4] T. R. Akash, N. Lessard, N. R. Reza, and M. S. Islam, “Investigating methods to enhance data privacy in business, especially in sectors like analytics and finance,” *Journal of Computer Science and Technology Studies*, vol. 6, no. 5, pp. 143–151, 2024.
- [5] A.-T. Tran, T.-D. Luong, and V.-N. Huynh, “A comprehensive survey and taxonomy on privacy-preserving deep learning,” *Neurocomputing*, p. 127345, 2024.
- [6] Y. Gao, H. Deng, Z. Zhu, X. Chen, Y. Xie, P. Duan, and P. Chen, “Peafowl: Private entity alignment in multi-party privacy-preserving machine learning,” *IEEE Transactions on Information Forensics and Security*, 2025.
- [7] L. Zhou, B. Zhang, Z. Wang, T. Lu, Q. Song, S. Zhang, H. Cui, and Y. Yu, “On probabilistic truncation in privacy-preserving machine learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 21, 2025, pp. 22 955–22 964.

- [8] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 19–38.
- [9] M. Al-Rubaie and J. M. Chang, "Privacy-preserving machine learning: Threats and solutions," *IEEE Security & Privacy*, vol. 17, no. 2, pp. 49–58, 2019.
- [10] S. E. V. S. Pillai and K. Polimetla, "Enhancing network privacy through secure multi-party computation in cloud environments," pp. 1–6, 2024.
- [11] R. L. Rivest, L. Adleman, M. L. Dertouzos *et al.*, "On data banks and privacy homomorphisms," *Foundations of secure computation*, vol. 4, no. 11, pp. 169–180, 1978.
- [12] G. Su, J. Wang, X. Xu, Y. Wang, and C. Wang, "The utilization of homomorphic encryption technology grounded on artificial intelligence for privacy preservation," *International Journal of Computer Science and Information Technology*, vol. 2, no. 1, pp. 52–58, 2024.
- [13] A. Gillioz, J. Casas, E. Mugellini, and O. Abou Khaled, "Overview of the transformer-based models for nlp tasks," in *2020 15th Conference on computer science and information systems (FedCSIS)*. IEEE, 2020, pp. 179–183.
- [14] A. D. Rasamoelina, F. Adjailia, and P. Sinčák, "A review of activation function for artificial neural network," in *2020 IEEE 18th world symposium on applied machine intelligence and informatics (SAMII)*. IEEE, 2020, pp. 281–286.
- [15] D. Demmler, T. Schneider, and M. Zohner, "Aby-a framework for efficient mixed-protocol secure two-party computation," in *NDSS*, 2015.
- [16] P. Mohassel and P. Rindal, "Aby3: A mixed protocol framework for machine learning," pp. 35–52, 2018.
- [17] V. Fang, L. Brown, W. Lin, W. Zheng, A. Panda, and R. A. Popa, "Costco: An automatic cost modeling framework for secure multi-party computation," in *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2022, pp. 140–153.
- [18] Q. Zhang, T. Xiang, C. Xin, and H. Wu, "From individual computation to allied optimization: Remodeling privacy-preserving neural inference with function input tuning," 2024.
- [19] —, "United we stand: Accelerating privacy-preserving neural inference by conjunctive optimization with interleaved nexus," vol. 38, no. 15, pp. 16 794–16 802, 2024.
- [20] T. Chen, H. Bao, S. Huang, L. Dong, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei, "THE-X: Privacy-preserving transformer inference with homomorphic encryption," in *Findings of the Association for Computational Linguistics: ACL 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3510–3520.
- [21] T. Xu, L. Wu, R. Wang, and M. Li, "Privcircuit: Efficient private inference via block circulant transformation," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 111 802–111 831. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/ca9873918aa72e9033041f76e77b5c15-Paper-Conference.pdf
- [22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [23] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.
- [24] Q. Pang, J. Zhu, H. Mollering, Z. Wenting, and T. Schneider, "Bolt: Privacy-preserving, accurate and efficient inference for transformers," 2024.
- [25] W.-j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, K. Ren, C. Hong, T. Wei, and W. Chen, "Bumblebee: Secure two-party inference framework for large transformers," *Cryptology ePrint Archive*, 2023.
- [26] D. Rathee, N. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow2: Practical 2-party secure inference," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 325–342.
- [27] J. Zhang, J. Liu, X. Yang, Y. Wang, K. Chen, X. Hou, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," *Cryptology ePrint Archive*, 2024.
- [28] E. Pattuk, M. Kantarcioglu, H. Ulusoy, and B. Malin, "Cheapsmc: A framework to minimize secure multiparty computation cost in the cloud," in *Data and Applications Security and Privacy XXX: 30th Annual IFIP WG 11.3 Conference, DBSec 2016, Trento, Italy, July 18-20, 2016. Proceedings 30*. Springer, 2016, pp. 285–294.
- [29] A. Schroepfer and F. Kerschbaum, "Forecasting run-times of secure two-party computation," in *2011 Eighth International Conference on Quantitative Evaluation of Systems*. IEEE, 2011, pp. 181–190.
- [30] E. Chen, J. Zhu, A. Ozdemir, R. S. Wahby, F. Brown, and W. Zheng, "Silph: A framework for scalable and accurate generation of hybrid mpc protocols," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 848–863.
- [31] F. Kerschbaum, T. Schneider, and A. Schröpfer, "Automatic protocol selection in secure two-party computations," in *Applied Cryptography and Network Security: 12th International Conference, ACNS 2014, Lausanne, Switzerland, June 10-13, 2014. Proceedings 12*. Springer, 2014, pp. 566–584.
- [32] O. Renaud and M.-P. Victoria-Feser, "A robust coefficient of determination for regression," *Journal of Statistical Planning and Inference*, vol. 140, no. 7, pp. 1852–1862, 2010.
- [33] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full rms variant of approximate homomorphic encryption," in *International Conference on Selected Areas in Cryptography*. Springer, 2018, pp. 347–368.
- [34] E. Lee, J.-W. Lee, J. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and W. Choi, "Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions," in *International Conference on Machine Learning*. PMLR, 2022, pp. 12 403–12 422.
- [35] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," pp. 1–23, 2010.
- [36] M. Naor and B. Pinkas, "Efficient oblivious transfer protocols," in *SODA*, vol. 1, 2001, pp. 448–457.
- [37] D. Rathee, M. Rathee, R. K. K. Goli, D. Gupta, R. Sharma, N. Chandran, and A. Rastogi, "Sirnn: A math library for secure rnn inference," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1003–1020.
- [38] H. Ghodosi, J. Pieprzyk, and R. Steinfeld, "Multi-party computation with conversion of secret sharing," *Designs, Codes and Cryptography*, vol. 62, pp. 259–272, 2012.
- [39] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shepsman, N. Vasilache, and O. Zinenko, "Mlir: A compiler infrastructure for the end of moore's law," *arXiv preprint arXiv:2002.11054*, 2020.
- [40] Y. Chen, Z. Tang, T. Lu, B. Zhang, Z. Shi, and Z. Wang, "Accelerating private large transformers inference through fine-grained collaborative computation," 2025. [Online]. Available: <https://arxiv.org/abs/2412.16537>
- [41] A. Ozdemir, F. Brown, and S. W. Riad, "Circ: Compiler infrastructure for proof systems, software verification," and more. Cryptology ePrint Archive, Paper 2020/1586, 2020. <https://eprint.iacr.org/2020/1586>, 2020. Tech. Rep., 2020.
- [42] A. C.-C. Yao, "How to generate and exchange secrets," in *27th annual symposium on foundations of computer science (Sfcs 1986)*. IEEE, 1986, pp. 162–167.
- [43] H. Kwon, H. M. Kim, C.-G. Lyuh, J.-K. Kim, J. Han, and Y. Kwon, "Ai-warek: Compiling pytorch model for ai processor using mlir framework," in *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2022, pp. 463–465.
- [44] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [45] Y. Hu, S. Yu, X. Qi, W. Zheng, Q. Wang, and H. Yao, "An overview of multiple linear regression model and its application," *Zhonghua yu Fang yi xue za zhi [Chinese Journal of Preventive Medicine]*, vol. 53, no. 6, pp. 653–656, 2019.
- [46] X. Ren, L. Su, Z. Gu, S. Wang, F. Li, Y. Xie, S. Bian, C. Li, and F. Zhang, "Heda: multi-attribute unbounded aggregation over homomorphically encrypted database," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 601–614, 2022.
- [47] H. Borchani, G. Varando, C. Bielza, and P. Larranaga, "A survey on multi-output regression," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 5, no. 5, pp. 216–233, 2015.
- [48] J. P. Vielma, "Mixed integer linear programming formulation techniques," *Siam Review*, vol. 57, no. 1, pp. 3–57, 2015.
- [49] X. Wang, A. J. Malozemoff, and J. Katz, "EMP-toolkit: Efficient MultiParty computation toolkit," <https://github.com/emp-toolkit>, 2016.
- [50] N. Chandran, D. Gupta, A. Rastogi, R. Sharma, and S. Tripathi, "Ezpc: Programmable and efficient secure two-party computation for machine

- learning,” in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 496–511.
- [51] “Microsoft SEAL (release 4.1),” <https://github.com/Microsoft/SEAL>, Jan. 2023, microsoft Research, Redmond, WA.
 - [52] Z. Huang, W. j. Lu, C. Hong, and J. Ding, “Cheetah: Lean and fast secure {Two-Party} deep neural network inference,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 809–826.
 - [53] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, “Iron: private inference on transformers,” *Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS ’22)*, p. 15718–15731, 2022.
 - [54] J. Ma, Y. Zheng, J. Feng, D. Zhao, H. Wu, W. Fang, J. Tan, C. Yu, B. Zhang, and L. Wang, “{SecretFlow-SPU}: A performant and {User-Friendly} framework for {Privacy-Preserving} machine learning,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 17–33.
 - [55] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.